

技術スタック分析レポート

AI Chat Fulemon | 会社向け詳細版

Next.js と Ollama を組み合わせた

ローカル AI チャットアプリの構成資料

エグゼクティブサマリー

- ローカル LLM による AI 応答
- トピック単位の履歴管理
- モデル選択と会話の継続
- 外部 API キー不要
- 段階的な機能拡張が可能

作成日: 2026年9月4日

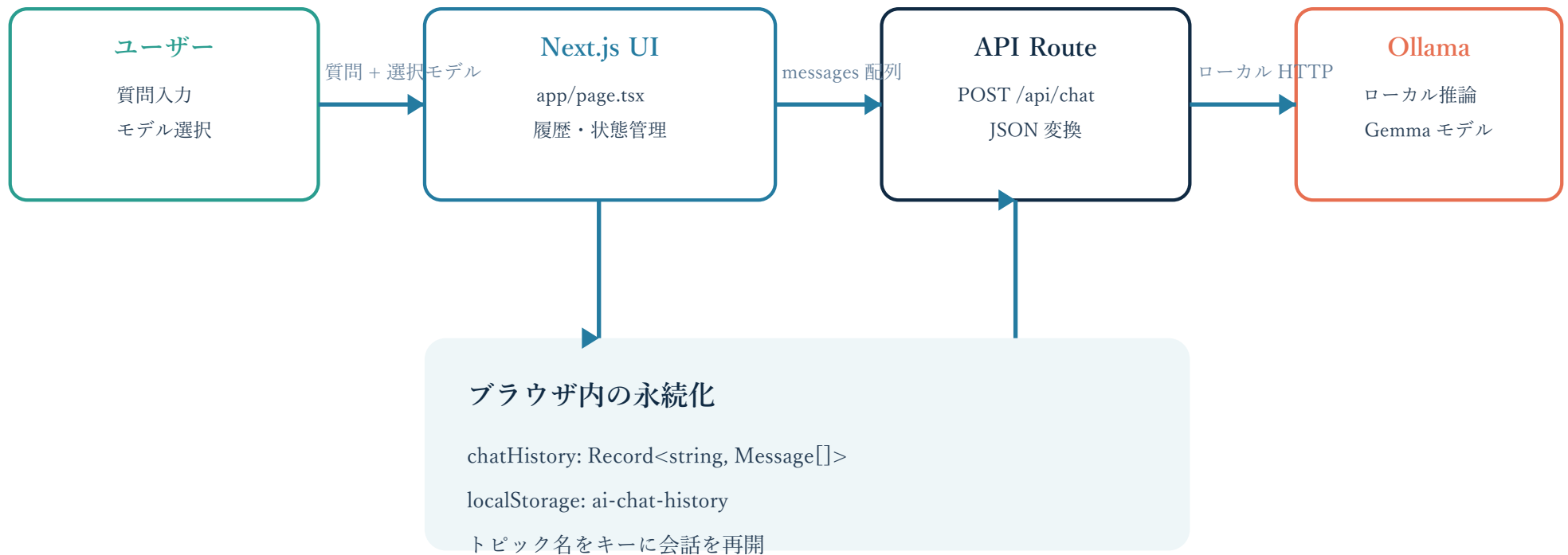
技術スタック一覧

主要な技術、実装箇所、採用理由を一覧化しています。

分類	技術・実装	役割 / 詳細
アプリ基盤	Next.js 16.3.4	App Router、画面配信、API ルートの基盤
UI	React 19.2.8	チャット画面、サイドバー、フォームの描画と状態更新
型・品質	TypeScript 5 / ESLint 9	型安全な実装とコード品質の維持
スタイリング	Tailwind CSS 4	レスポンスなレイアウトと UI スタイル
API	Next.js Route Handler	app/api/chat/route.ts で POST リクエストを処理
AI 実行基盤	Ollama API	localhost:11434/api/chat へ会話履歴とモデル名を送信
モデル	Gemma4 e2b / e4b	軽量モデルと高性能モデルを UI から切り替え
状態管理	React useState	入力、選択モデル、ローディング、現在トピックを管理
永続化	ブラウザ localStorage	Record<string, Message[]> 形式でチャット履歴を保存
履歴機能	トピック CRUD	新規作成、質問からの自動命名、編集、削除、再開

アーキテクチャ図

質問が入力されてから、ローカル LLM の回答が画面へ戻るまでの処理フロー。



詳細分析

設計方針

フロントエンドと API を同一の Next.js プロジェクトにまとめ、ローカル LLM とブラウザ保存を組み合わせている。外部 API キーを使わず、開発・検証・社内利用を始めやすい構成である。

リクエスト処理

ユーザーの入力と選択モデルをクライアントから API ルートへ送信する。API ルートは Ollama のチャット API に履歴を渡し、生成された応答テキストを JSON として画面へ返す。

データモデル

チャット履歴はトピック名をキー、Message 配列を値とする Record として管理する。各トピックには system、user、assistant のメッセージが時系列で保存される。

拡張ポイント

認証、サーバー側データベース、ストリーミング応答、利用ログ、権限管理を追加する場合も、API ルートと履歴管理を境界として段階的に拡張できる。

主なファイル構成

app/page.tsx	メインチャット UI、履歴管理、モデル選択
app/api/chat/route.ts	Ollama への API プロキシ
app/layout.tsx	アプリ全体のレイアウト
app/globals.css	グローバルスタイル
package.json	依存関係と実行スクリプト

まとめと今後の拡張候補

現時点の構成は、ローカル AI チャットを短い導入期間で検証するために適している。フロントエンド、API、LLM 実行基盤の責務が分かれているため、将来の本番運用にも段階的に移行できる。

短期 ストリーミング応答、入力バリデーション、エラー表示の改善

中期 SQLite / PostgreSQL などへの履歴移行、ユーザー認証、利用ログ

長期 権限管理、チーム共有、監査ログ、モデル管理画面

総括 Next.js、React、TypeScript、Tailwind CSS、Ollama を組み合わせた、拡張性のあるローカル AI チャット基盤です。

